

WindowMirror: an opensource toolkit to bring interactive multi-window views into XR

Riccardo Bovo*
Imperial College London
Google

Eric J. Gonzalez
Google

Li-Te Cheng
Google

Mar Gonzalez-Franco†
Google

ABSTRACT

WindowMirror¹ toolkit provides users with a simulated, extended screen real-estate. It allows users to interact with multiple desktop applications in real-time within a XR environment. Our architecture has two main modules: a Unity package and a Python backend. WindowMirror supports traditional desktop interaction methods such as mouse, keyboard and it features a Cylindrical Window Layout, an emerging design pattern which is particularly effective for single-user, egocentric perspectives.

Index Terms: Human-centered computing—Visualization—Visualization techniques—Treemaps; Human-centered computing—Visualization—Visualization design and evaluation methods

1 INTRODUCTION

Virtual Reality (VR) holds the potential to unlock knowledge work to the next level by extending 2D displays into a much larger real-estate [3]. Unlike physical screens, which require space, are expensive, and lack portability, virtual screens are free from these constraints [16]. Furthermore, virtual screen environments can facilitate work-from-home, and on-the go arrangements by addressing issues such as distractions, inadequate workspace, and the challenge of maintaining work-life balance [6]. As an increasing number of people work during commutes, VR screens open up new possibilities and challenges in the realm of mobile office work [15].

These factors contribute to the growing belief that VR screens will transform the way information workers operate, offering low-cost, personalized, and private working environments, even while on the move. As a result multiple HCI studies have explored the use of VR flat screen pannels. Hope et al. [9] proposed a VR application that mirrors desktop applications on 2D screen in the VR environment. Several studies have explored the ideal placement of screens in the 3D environment, notably the work of Satriadi et al [17] explored the 3D placement of 2D surfaces for individual tasks, while the work of Lee et al. [11] looked at it from the angle of collaborative environment for presentations tasks. Other notable explorations are related to visual attention cues in VR screens environments [4, 5] and neck movement minimization [12].

Interestingly, it is only very recently that we have seen platforms like Horizon Workrooms (2021) [1, 2], which offer virtual spaces for traditional desktop work, including remote desktop capabilities, despite the long-anticipated future of productivity within VR headsets. A side effect is that despite the increased interest in Virtual Reality (VR) and Human-Computer Interaction (HCI) research in this space, there is still a lack of open-source applications capable of mirroring multiple independent and interactive windows at full resolution. While there are some proprietary application that enable users to mirror arbitrary application from a Linux environment [13, 18, 20]

*e-mail: rb1619@ic.ac.uk

†e-mail: margon@google.com

¹<https://github.com/google/window-mirror>

there are no open source applications which can easily be integrated with development platforms such as Unity.

This gap poses a challenge for HCI researchers in developing and testing prototypes. To address this issue, we introduce WindowMirror, a tool that comprises a Unity package and a Python module which can be downloaded from GitHub (<https://github.com/google/window-mirror>). This application enables experimenters to quickly and easily set up a desktop mirroring system with minimal technical knowledge required.

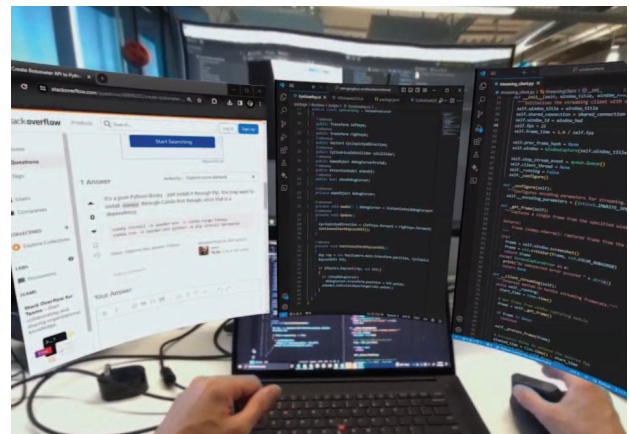


Figure 1: Depiction of the application displaying two VR screen windows mirrored from a laptop.

2 WINDOWMIRROR ARCHITECTURE

The WindowMirror system architecture (Figure 3) is composed of two main applications: the first runs on the PC. It is a streaming application, written in Python, which handles the capturing and streaming of windows using the Win32 API. The second is a Unity VR application that generates and arranges these windows within the 3D space of the VR headset, and manages the input interactions.

Our system uses JPEG images that are captured and transmitted from the Window Capturing module of the Python application on the PC to the Window Display module of the Unity VR application via socket on a local private network. User interface events are captured within the Unity VR application and relayed back to the Event Simulation module of the Python application. This setup enables a Windows application to be mirrored and interacted with in a VR environment, thus creating a fully interactive user interface to productivity scenarios in VR.

Python application in Windows can be abstracted in 3 modules: a window capturing module, an event simulation module and a streaming module.

- **Window Capturing:** This module is responsible for capturing the contents of a window. It uses the Win32 API to capture the screen and converts this capture to a JPEG image using

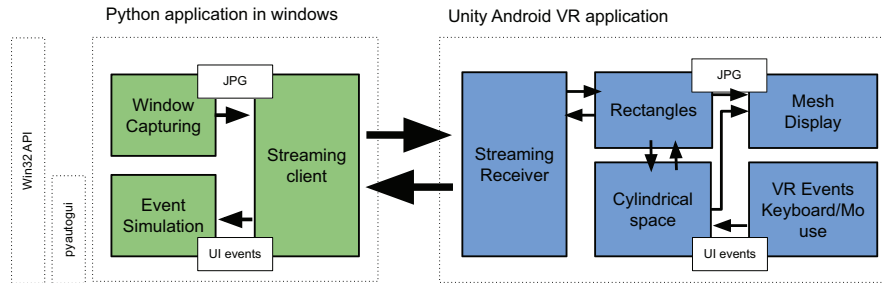


Figure 2: The system architecture is composed of two main applications: the first is a streaming application, written in Python, and the second is a Unity VR application that generates and arranges these windows within the 3D space of the VR headset, and manages the input interactions. The application communicates via socket across a local network.

OpenCV. The frame rate is set a-priori however the system avoid sending updates if there are not differences in the frames therefore enabling achieving a variable frame rate which depend on events. The choice of JPEG compression using images instead of a modern video compression algorithms is deliberate aim to keep the simplicity of the system allowing for not to technical user to extend or code around it.

- **Event Simulation:** This module simulates user interface events, such as mouse clicks or keyboard presses, it utilizes the pyautogui which is a wrapper for the Win32 API.
- **Streaming Client (Python):** This triggers the generation of the JPEG Window Capturing module and streams it to the Unity VR application via the local network. It also receives UI events from the Unity VR application to be simulated on the Windows side.

The Unity VR application is device agnostic and can be abstracted on to: a streaming receiver module, a cylindrical space module, a mesh display module, and a VR Events Keyboard/Mouse modules.

- **Streaming Receiver (Unity):** This component receives the JPEG stream from the Streaming Client in the Python application. The Receiver creates a novel Rectangle for each new window id received. A rectangle is an abstraction of a window that is then converted into 3D coordinates using the Cylindrical space module which effectively projects the rectangles into the cylindrical surface.
- **Cylindrical Space:** this is where the images are mapped onto a curved surface, so that each rectangle corresponds to a Mesh Display.
- **The Mesh display module** receives JPEG images and updates the surface textures, allowing the user to view and interact with the mirrored application windows.
- **VR Events Keyboard/Mouse modules** capture the user's interactions with the cylindrical space, such as keyboard and mouse events, the UI events are remapped from 3D coordinates to normalized cylindrical coordinates to px coordinates and are sent back back to via the Receiver to the Streaming Client in the Python application to simulate the user's actions within the Windows environment.

3 DESIGN CONSIDERATIONS

To enable interactivity on the Unity end, we enabled support for mouse and keyboard inputs, that can be seamlessly integrated with

Android-based headsets and are also accessible within the Unity editor. The open-sourced tool only includes the inclusion of the screens into an empty Unity project and no actual HMD dependencies. Our application is device agnostic so it aims to be compatible both with pass-through and with tracked mouse and keyboard in full VR if pass through is not available.

However, our actual tests and implementation inside HMDs, we found that the optimal solution was a pass-through environment approach. This allows users to simultaneously view both the hands, the keyboard and the mouse in their actual form, eliminating the need to track and represent them as 3D models within an immersive environment.

3.1 Cylindrical Layout

The field of VR for productivity is rapidly evolving, and it appears that VR screen window management systems are increasingly adopting an egocentric cylindrical layout. This particular window manager design has been widely recognized and utilized in various research papers for both single-user productivity scenarios [3, 12, 16] and multi-user productivity [4, 11, 17]. It has also been incorporated into mainstream software applications [10, 14] and recommended in platform guidelines [7]. In light of this trend, we have implemented an automatic window placement system that leverages a parametric cylindrical space. This design allows researchers to easily adjust its radius and height to tailor the system to their specific applications, thereby enhancing functionality and user experience in virtual environments.

4 EVALUATION

One key aspect to evaluate on screen mirroring is the frames per second that can be run on the system [8]. And while this will be highly dependent on the network as well as HMD compute, we have done preliminary tests using existing online tools (<https://fpstest.org/>) on optimal conditions (aka Unity and python app running on the same PC, and no HMD connected) and found the system achieved over 7 fps on a screen with a full resolution of 1650x 1050. This highlights one limitation to the toolkit will be that it will not be able to run videos, and it has screen casting limitations. However, for normal productivity tasks, the fps limitations were not perceivable by users.

With regards to resolution of the windows they will be limited to the resolution of the screen on the PC that is running the back-end. Having many windows at high resolution open could also affect fps, our preliminary tests running only on CPU showed that the system run well up to 4-6 screens on a 11th Gen Intel (R) Core (TM) i7-11850 @ 2.5 Hz with a 32GB RAM and x64 processor, but fps would decay after. Specially when both the Unity and the Python code were running on the same machine.

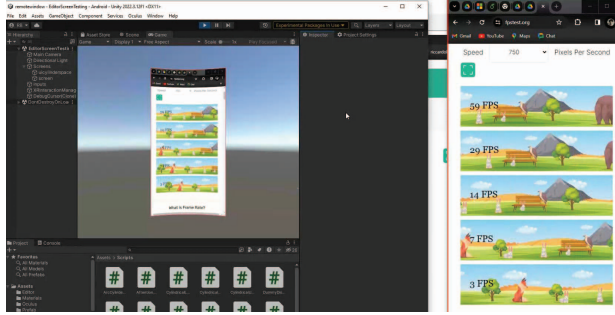


Figure 3: The frames per second on the system were tested preliminary using online tools (<https://fpstest.org/>) over 7 fps on a screen with a full resolution of 1650×1050 .

5 CONCLUSIONS AND FUTURE WORK

The primary features of our WindowMirror framework include its simplicity, comprehensive mouse and keyboard interactivity support, and an egocentric cylindrical layout, making it a valuable tool for researchers in the field.

Future work should aim to enhance the system's capabilities while maintaining its simplicity. For example, this could include the integration of overlays for finer textures will significantly improve visual quality. The improvement on FPS so the system can be used for video too, could be done perhaps using a tool like gStreamer, which can achieve network speeds for video of over 28–30 frames per second encoding speed at $1,600 \times 1,080$ resolution [19]. Additionally fps could also increase, and hence also the number of open windows

From the HCI perspective, additional work could be done on exploring alternative mapping modalities beyond the current cylindrical approach. The current implementation should provide researchers examples to follow to create their own screen mapping version.

Further advancements on this toolkit will be approached with a practical mindset, prioritizing simplicity for researcher-centring improvements that align with the evolving needs and expectations of XR productivity research. In conclusion, our aspiration is that this work serves as a solid foundation for future research in XR screen-focused productivity scenarios, laying the groundwork for innovative developments in this field.

REFERENCES

- [1] Horizon Workrooms, 2021. Virtual Collaboration Software.
- [2] L. Aufegger and N. Elliott-Deflo. Virtual reality and productivity in knowledge workers. *Frontiers in Virtual Reality*, 3:890700, 2022.
- [3] V. Biener, D. Schneider, T. Gesslein, A. Otte, B. Kuth, P. O. Kristensson, E. Ofek, M. Pahud, and J. Grubert. Breaking the screen: Interaction across touchscreen boundaries in virtual reality for mobile knowledge workers. *IEEE transactions on visualization and computer graphics*, 26(12):3490–3502, 2020.
- [4] R. Bovo, D. Giunchi, M. Alebri, A. Steed, E. Costanza, and T. Heinis. Cone of vision as a behavioural cue for vr collaboration. *Proceedings of the ACM on Human-Computer Interaction*, 6(CSCW2):1–27, 2022.
- [5] R. Bovo, D. Giunchi, L. Sidenmark, J. Newn, H. Gellersen, E. Costanza, and T. Heinis. Speech-augmented cone-of-vision for exploratory data analysis. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, pp. 1–18, 2023.
- [6] N. Fereydooni and B. N. Walker. Virtual reality as a remote workspace platform: Opportunities and challenges. 2020.
- [7] Google Developers. Google daydream ui guidelines. <https://youtu.be/ES9jArHRFHQ>, 2017. Accessed: 1/9/2024.
- [8] R. Gruen, E. Ofek, A. Steed, R. Gal, M. Sinclair, and M. Gonzalez-Franco. Measuring system visual latency through cognitive latency

on video see-through ar devices. In *2020 IEEE Conference on Virtual Reality and 3D User Interfaces (VR)*, pp. 791–799. IEEE, 2020.

- [9] A. H. Hoppe, F. van de Camp, and R. Stiefelham. Enabling interaction with arbitrary 2d applications in virtual environments. In *HCI International 2020-Posters: 22nd International Conference, HCII 2020, Copenhagen, Denmark, July 19–24, 2020, Proceedings, Part II 22*, pp. 30–36. Springer, 2020.
- [10] Immersed Developers. Immersed, 2022. Accessed: 1/9/2024.
- [11] B. Lee, X. Hu, M. Cordeil, A. Prouzeau, B. Jenny, and T. Dwyer. Shared surfaces and spaces: Collaborative data visualisation in a co-located immersive environment. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):1171–1181, 2020.
- [12] M. McGill, A. Kehoe, E. Freeman, and S. Brewster. Expanding the bounds of seated virtual workspaces. *ACM Transactions on Computer-Human Interaction (TOCHI)*, 27(3):1–40, 2020.
- [13] Nova King. Stardust xr. <https://stardustxr.org/>, 2022. Accessed: 1/9/2024.
- [14] Oculus Developers. Oculus infinite office. https://youtu.be/5_bVkbG1ZCo, 2022. Accessed: 1/9/2024.
- [15] E. Ofek, J. Grubert, M. Pahud, M. Phillips, and P. O. Kristensson. Towards a practical virtual office for mobile knowledge workers. *arXiv preprint arXiv:2009.02947*, 2020.
- [16] L. Pavanatto, C. North, D. A. Bowman, C. Badea, and R. Stokley. Do we still need physical monitors? an evaluation of the usability of ar virtual monitors for productivity work. In *2021 IEEE Virtual Reality and 3D User Interfaces (VR)*, pp. 759–767. IEEE, 2021.
- [17] K. A. Satriadi, B. Ens, M. Cordeil, T. Czauderna, and B. Jenny. Maps around me: 3d multiview layouts in immersive spaces. *Proceedings of the ACM on Human-Computer Interaction*, 4(ISS):1–20, 2020.
- [18] Simula VR. simula vr. <https://simulavr.com/>, 2022. Accessed: 1/9/2024.
- [19] N. Smolyanskiy and M. Gonzalez-Franco. Stereoscopic first person view system for drone navigation. *Frontiers in Robotics and AI*, p. 11, 2017.
- [20] Zwin. zwin. <https://www.zwin.dev/>, 2022. Accessed: 1/9/2024.